

Setup Time Prediction Using Machine Learning Algorithms: a Real-world Case Study

Alberto Locatelli¹[0000–0002–5368–8289], Manuel Iori¹[0000–0003–2097–6572],
Marco Lippi¹[0000–0002–9663–1071], and Marco Locatelli²[0000–0001–7138–8653]

¹ Department of Sciences and Methods for Engineering, University of Modena and Reggio Emilia, Reggio Emilia, Italy,

{manuel.iori, marco.lippi, alberto.locatelli}@unimore.it.

² Department of Engineering and Architecture, University of Parma, Parma, Italy,
marco.locatelli@unipr.it.

Abstract. In this paper, we explore the use of machine learning regression algorithms for setup time prediction and we apply them to a real-world scheduling application arising in the color printing industry. As the complexities associated with setup time predictions have received limited attention from the literature, we aim at exploiting a data-driven approach based on machine learning algorithms to enhance the quality of setup time evaluations and narrow the gap between scheduling theory and practice. Using a real-world industrial dataset, we train three different machine learning models: linear regression, random forests, and gradient boosting machines. The experimental results demonstrate that the gradient boosting machine approach obtains the best performance overall, immediately followed by random forests. The accuracy of the obtained predictions shows the effectiveness of the proposed approach in setup time evaluation. The obtained results are particularly significant and valuable due to the versatile nature of the proposed machine learning approaches. These methods can be applied to various scheduling scenarios, making them suitable for integration into scheduling algorithms to potentially improve their accuracy.

Keywords: Setup time prediction, Machine learning algorithms, Sequence-dependent setup costs, Color printing industry, Food industry

1 Introduction

According to Allahverdi and Soroush [1], *setup time* (ST) can be defined as the time required to prepare the necessary resource (e.g., machines, people) to perform a task (e.g., a job, an operation). Setup operations may include, for instance, switching tools, cleaning up, changing material, adjusting tools, etc. As all these operations are often strongly time-consuming and may take a considerable part of the entire production time, the reduction of STs plays a crucial role in scheduling. The deep connection between scheduling activities and STs was analyzed in detail by Allahverdi [2], who provided a comprehensive survey on scheduling problems involving STs.

Although in many real-world manufacturing environments STs are influenced by various and random factors (e.g., crew experience, breakdowns of a tool or a machine, lack of personnel, complex and non-fixed procedures, etc.), in the extant literature, the vast majority of scheduling studies consider STs as a given input taking a deterministic value, calculated by means of simplistic average-based methods that are not capable to catch the complexity of reality. As already highlighted by Kim and Bobrowski [3], assuming stochastic STs as fixed and constant values may lead to develop inefficient schedules. This is also confirmed by practitioners, who often view the lack of uncertainty and dynamic elements in the modeling of scheduling processes as the major source of gap between scheduling theory and practice (see Sabuncuoglu and Goren [4]). Only few studies, recently reviewed by Allahverdi [5], took into account the issues related to the STs uncertainty. These studies mainly focused on robust optimization approaches (modeling STs as a probability distribution, or a fuzzy number, or a random variable within some interval), but did not provide any methods to estimate STs in practice.

In this work, we seek to fill this gap of the literature by providing models that can be used to provide good estimations of the STs starting from a set of empirical data. In detail, we use *machine learning* (ML) regression algorithms to predict the STs, and we apply them to a real-world scheduling application arising in the color printing industry, where a finite set of jobs must be sequentially performed by a heterogeneous set of parallel flexographic printer machines. Specifically, we deal with *uncertain machine-dependent and job sequence-dependent setup times* (UMJSSTs) with an additional issue: the UMJSST between two jobs not only depends on the two jobs and the involved machine, but also on all jobs previously scheduled on that specific machine, owing to tool configurations (see, e.g., Soares and Carvalho [6]). Indeed, jobs have different tooling requirements and it is often beneficial to leave a certain tool unused in a machine magazine only to use it again a few jobs later (see Iori et al. [7, 8, 9]). We use the acronym UMJESTs to denote UMJSSTs with such job entire-sequence-dependent characteristic.

In the addressed real-world industrial application, UMJESTs are strongly time-consuming (around 65% of the total production time) and subject to significant uncertainties, because depending on many factors: characteristics of the specific machine, status of the machine (which directly depends on the jobs already processed), characteristics of the current job, operators' experience, etc. It turns out that the problem of predicting such times is not trivial and may require a large computational effort (see Iori et al. [7]). Iori et al. [8] introduced a *heuristic evaluation method* (EEM), which expresses UMJESTs in terms of some specific and pre-fixed features (i.e., characteristics of jobs and machines). This analytic approach suffers from intrinsic limitations and cannot account for the whole complexity of the problem.

To improve the accuracy of UMJESTs prediction, we aim at exploiting a data-driven approach based on ML algorithms, to be exploratory both in selecting the significant features and expressing the UMJESTs in terms of these features. Using a real-world industrial dataset, we train three different ML models: *linear*

regression (LR), *random forest* (RF), and *gradient boosting machine* (GBM). For each model, we take into account a wide set of features and several possible interdependencies among them, in order to identify a parsimonious but comprehensive subset of these features. We compare the three models with EEM by Iori et al. [8] on a real-world industrial test set. The experimental results demonstrate that the GBM approach obtains the best performance overall, immediately followed by RF. For both models, the mean squared error on the predicted UMJESTs is less than half of that of EEM, proving their effectiveness in modeling the application. The results are of interest because the ML models can be easily adapted to deal with ST evaluation in many other scheduling problems.

2 Brief Literature Review

Almost the totality of scheduling studies that took into account issues related to the STs uncertainty mainly focused on providing robust optimization methods to best manage the shortcomings in the STs prediction (see González-Neira et al. [10]). In this context, the uncertainty of STs is generally represented by interval data or fuzzy description. Aydilek et al. [11, 12], Allahverdi and Allahverdi [13], and Allahverdi [14] modeled sequence-independent STs by using interval data, while Yimer and Demirli [15] and Lu et al. [16] represented sequence-independent and sequence-dependent family STs, respectively, by fuzzy sets. To represent the shortcomings associated with estimation of sequence-dependent STs, Lu et al. [16] and Naderi-Beni et al. [17] used fuzzy description, while Behnamian and Fatemi Ghomi [18] used interval description.

As sequence-dependent STs are subject to many complexities, the problem of predicting such times is not trivial and, as already highlighted in Section 1, standard analytical approaches cannot account for the whole complexity of the problem. Surprisingly, advanced analytic methods received limited attention in this context. To the best of our knowledge, the only study that dealt with STs estimation complexities is the one by Cheng et al. [19], who proposed a random-forest-based metaheuristic to minimize the makespan in an unrelated parallel machines scheduling problem with UMJSSTs.

In contrast to the traditional sequence-dependent STs, the UMJESTs between two jobs depend also on all jobs previously scheduled on the involved machine, rather than only on the two jobs. This characteristic of STs typically arises in the context of flexible manufacturing systems (see, e.g., Chryssolouris [20]), where a flexible manufacturing machine (see, e.g., Crama [21]) is capable to handle different types of operations performed by the available tools installed in its limited capacity magazine. As each job requires a subset of tools and the magazine cannot hold all tools at once, some tool switches may be necessary when performing two jobs in succession (see, e.g., Laporte et al. [22]). If a tool switch is required, then a positive setup time is incurred. Thus, it is often beneficial to leave a certain tool unused in the magazine to use it again a few jobs later (see, e.g., Soares and Carvalho [6]). This feature of UMJESTs adds further complexities and uncertainties to STs and makes the problem of predicting UM-

JESTs even more challenging to solve in practice. To solve practical industrial instances, Iori et al. [8] introduced EEM, a fast evaluation method for predicting UMJESTs. EEM first determines the required setup operations based on a specific policy and then calculates UMJESTs by summing all the times required for each operation using standard times, which are calculated based on the historical average values for each operation. This simplistic approach is not capable to incorporate the whole complexity of reality and, in addition, does not take into account any issues related to the STs uncertainty. In this paper, to improve the accuracy of UMJESTs prediction, we employ three different ML techniques: LR, RF, and GBM. There is a vast literature on these three ML algorithms and we refer the interested reader to the comprehensive books by Montgomery et al. [23], Breiman et al. [24], and to the article by Natekin and Knoll [25], respectively.

3 Methodology

Firstly, we formulate the task of forecasting UMJESTs as a regression problem, where the aim is to predict a UMJEST, represented by a real positive number $y \in \mathbb{R}$, as a function f of a n -dimensional set A of attributes/features. We recall that the UMJEST between two jobs depends on many factors: the involved machine, the two jobs, and all jobs previously scheduled on that specific machine, thus the attributes in A must express all these factors. On the other hand, a too wide set of attributes may lead to overfitting issues and uncertainty predictions, reducing the effectiveness of the ML models (see Kuhn and Johnson [26]). Therefore, we initially consider a large set of features, which we subsequently refine by analyzing the inter-dependencies amongst them, and by using a feature selection method based on the importance scores generated by RF. In this way, we identify a parsimonious but comprehensive subset of the initial features.

As already stated in Section 1, we use supervised ML algorithms to build mathematical models for predicting UMJESTs. In this phase, a finite dataset $\mathcal{D}_{train} = \{(x_i, y_i)\}_{i=1}^m$ of pairs, each containing both the n -dimensional input vector of features $x_i = (x_i^1, \dots, x_i^n) \in A$ and the desired output $y_i \in \mathbb{R}$, is provided. The learning task is to find a function $\tilde{f}$ belonging to a certain family F of machine learning models (in our case LR, RF, and GBM) that, for each input x_i , has a predicted output $\tilde{y}_i = \tilde{f}(x_i)$ as close as possible to the provided target y_i .

As first basic approach, we compute f as a linear combination of input features $f(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$ with the vector of parameters $(\beta_0, \dots, \beta_n)$ that we wish to learn. LR is a widely employed technique that achieves good results when a linear function can represent a reasonable approximation of the relation holding between input and output variables (see Lewis-Beck and Lewis-Beck [27]).

In our problem there is a complex non-linear relation between target and heterogeneous variables (nominal, ordinal, and scalar), thus we also consider two non-linear models largely used in ML applications, namely RF and GBM. These

are able to manage heterogeneous data. More specifically, a RF is an ensemble learning method made up of a number of decision trees, called estimators, each of which is a weak prediction model with limited predictive power. However, when combined, a RF model leverages the diversity of these weak models to produce a more accurate prediction by aggregating their individual predictions (see Breiman [28]). Similarly, a GBM is an ensemble of weak prediction models (in our case decision trees) combined into a single strong learner in an iterative fashion. As each weak learner is added, a new model is fitted to provide a more accurate estimate of the response variable. The new weak learners are maximally correlated with the negative gradient of the loss function, associated with the whole ensemble (see Hastie et al. [29]).

The aim of this work is to provide a general-purpose ML framework for predicting sequence-dependent STs. This is reflected in the choice of using general-purpose ML models, without resorting to more sophisticated and case-specific approaches difficult to replicate and adapt to other cases. More specifically, the choice fell on RF and GBM because, among the classic ML algorithms, their characteristics fit well with those required by the problem. Indeed, RF and GBM perform well with a high number of heterogeneous features and can be applied also when the data contain missing values, typical characteristics of a real-world industrial dataset. An additional important feature of RF and GBM is the evaluation of the relevance of a variable by means of measures, which provides a greater readability of the models and allows the identification of possible redundancy among features.

Finally, we compare the three obtained models, with the aim of finding the one that performs the best on our real-world dataset. The final unbiased generalization error of the selected model is computed on a provided test set.

4 Case and Data Description

We apply the general-purpose ML framework introduced in Section 3 to a real-world scheduling application arising in the color printing industry. The problem consists in evaluating UMJESTs between jobs that are processed by a set of heterogeneous flexographic printer machines along a scheduling period from January 2019 to October 2022.

4.1 Case Description

Flexography is a highly versatile printing technology frequently utilized in the packaging industry due to its ability to print on a wide range of materials, including plastics, cellophane, paper, and aluminum foil [30]. A flexographic printing machine is a flexible manufacturing machine [21] designed to perform various operations, such as coloring, embossing, and perforating, using different tools that are stored in its magazines. The number of tools a machine can handle is limited by the size of its magazine, and the number of ink cartridges that can be loaded at once is determined by the number of color groups.

The setup phase involves preparing the machine for printing a new job and may require performing some tool-switching operations. Indeed, the limited magazine capacity of the machines and the need to process various types of jobs often require replacing the currently installed tools with others required for the next jobs. As a wide range of colors is needed for jobs, changing the ink cartridges in the color groups of a machine is by far the most common switching operation performed between two jobs. This operation necessitates a thorough washing process to eliminate any previous color residue from the inner surface of the color group. Residues can potentially compromise the quality of the next job, and therefore, it is crucial to ensure that the color group is completely clean before proceeding with the next task. The length of time required for the color group washing process depends on the specific colors involved. In addition, the setup time is also contingent on the characteristics of the printer machine and the job in question. If a printer machine has an automatic washing system, the color groups can be washed in parallel, but if not, they must be washed one by one. The time required for setup also depends on the tools available in the magazine (i.e., ink cartridges, embossing rollers, and perforating rolls) and the tools required for the next job.

4.2 Data Description

The dataset $\mathcal{D}$ is provided by the company as an extraction of orders from the Company Management System database. In this format, each sample, corresponding to a processed job, has 74 attributes (e.g., job *id*, machine *id*, tools required by the job, list of colours required by the job, *id* of the job processed right before by the same machine, etc.), many of which are removed because irrelevant for the prediction problem (e.g., order quantity, due date, release date, etc.). Table 1 reports the set of 22 attributes that may impact the UMJESTs.

To ensure the accuracy and consistency of the dataset, as suggested by the company, we only considered samples where the setup phase did not extend over two consecutive days. This decision was made due to the limitations of the system adopted by the company to detect the UMJESTs, which can only provide information about the start and end times of the setup phase. Consequently, the UMJESTs, calculated as the time difference between these two timestamps, are not available if the setup phase started during the day, paused overnight, and resumed the following day. This lack of information arises because we do not have precise details on when the operators paused the machine at the end of the daily shift and when they resumed work the next day, as the operators have flexible shifts and can work overtime. This issue has motivated the company to invest in a specialized *manufacturing execution system* (MES) to acquire improved and highly accurate data, enhancing also the proposed ML approaches. The initial dataset consists of a set of 9401 samples collected between January 2019 and October 2022.

Table 1: Features of dataset $\mathcal{D}$

Feature	Description	Type
F_1	Identification code of the job	Int
F_2	Identification code of the job processed right before	Int
F_3	Identification code of the set of required colours	Int
F_4	Identification code of the type of printed material	Int
F_5	The number of required colors	Int
F_6	1 st colour required	String
$\vdots$	$\vdots$	$\vdots$
F_{15}	10 th colour required	String
F_{16}	Print mode, i.e, mirror or normal print	String
F_{17}	If perforating roller is required	Boolean
F_{18}	Identification code of the required special varnish	Int
F_{19}	Identification code of the required embossing roller	Int
F_{20}	Identification code of the required corona treatment	Int
F_{21}	If job requires compostable inks	Boolean
F_{22}	Identification code of the machine used to process the job	Int

4.3 Pre-processing and Cleaning Phase

In this section, we outline the pre-processing and manipulation procedures applied to dataset $\mathcal{D}$ to enhance its quality, structure, and information content, with the aim of improving the performance of machine learning models. The features of the refined dataset are listed in Table 2.

Firstly, we detected and removed outliers by using a boxplot-based method [31]. After this cleaning procedure, we removed 4% of the samples from the dataset, reducing it from 9401 to 9007 samples.

As UMJESTs depend on many factors, the features reported in Table 1 are not capable to provide a complete representation of the information necessary for an accurate prediction of the UMJESTs. Thus, as a first step, we enriched the dataset by introducing additional features.

The results of a preliminary statistical analysis showed that UMJESTs are influenced by some specific characteristics of the machine used to process a job: i.e., the number of color groups on the machine and whether the machine has an automatic washing system. More precisely, when the machine has an automatic washing system, the color groups can be washed simultaneously; otherwise, they must be washed sequentially. Thus, feature F_{22} in Table 1 was split into three separate features reporting such additional information, that became features f_1 , f_2 , and f_3 in the new feature set described Table 2).

Due to the sequence-dependent nature of the UMJESTs, we replaced feature F_2 reporting the identification code of the job processed right before with a new set of features f_{60} - f_{116} (see Table 2) describing all the job characteristics.

After statistical analysis, we identified four tools for which the involving switching operations are mostly time-consuming: namely, white cartridge, varnish cartridge, perforating roller, and embossing roller. Since the UMJEST depends on the whole sequence, for each of these tools we added an additional feature (f_{117} - f_{119} in Table 2) which indicates whether a switching operation involving such a tool is required or whether that tool is already held in the mag-

Table 2: Features of the dataset $\mathcal{D}$ after the pre-processing and manipulation phase.

Feature	Description	Type
f_1	Identification code of the machine used to process the job	Int
f_2	Whether the machine has an automatic washing system	Boolean
f_3	The number of color groups of the machine	Int
f_4	The number of colors required by the job	Int
f_5	Identification code of the required embossing roller	Int
f_6	Whether perforating roller is required	Boolean
f_7	Identification code of the required special varnish	Int
f_8	Whether job requires compostable inks	Boolean
f_9	Identification code of the required corona treatment	Int
f_{10}	Print mode, i.e, mirror or normal print	String
f_{11}	Identification code of the type of printed material	Int
f_{12}	Identification code for the whole set of required colours.	Int
f_{13}	Whether colour red is required.	Boolean
$\vdots$	$\vdots$	$\vdots$
f_{59}	Whether colour yellow is required.	Boolean
f_{60}	Print mode of previous job (i.e, mirror or normal)	String
f_{61}	Identification code of the type of printed material of previous job	Int
f_{62}	Identification code of the set of required colours of previous job	Int
f_{63}	Whether colour red is required by previous job	Boolean
$\vdots$	$\vdots$	$\vdots$
$\vdots$	$\vdots$	$\vdots$
f_{116}	Whether colour yellow is required by previous job	Boolean
f_{117}	Whether a switch involving white cartridge is necessary.	Boolean
f_{118}	Whether a switch involving varnish cartridge is necessary.	Boolean
f_{119}	Whether a switch involving embossing roller is necessary.	Boolean
f_{120}	Whether f_{12} is equal to f_{62} .	Boolean

azine, considering also all the jobs previously processed by the involved machine. On the other hand, feature f_{120} indicates whether the current job and the one processed right before require exactly the same colours.

The transformation of categorical features F_5 - F_{16} into Boolean features f_{13} - f_{59} was implemented to simplify the representation of the data and improve its suitability for use by machine learning algorithms. Indeed, this conversion process not only simplifies the representation of the data, but also enhances the performance of the algorithms by reducing the complexity of the input space and making it more interpretable.

4.4 Feature Selection

We used the feature importance scores generated by RF to identify the most important features of the dataset. More precisely, feature importance is computed based on the average decrease in impurity over all trees in the forest. The higher the importance score of a feature, the more it was used to split the data across decision trees. We use a threshold parameter $\alpha \in [0, 1]$, to select only the features with importance scores above α . Then, we train the models using only this reduced set of features.

5 Experiments

This section illustrates the computational experiments conducted to assess the effectiveness of the proposed models and compare them with EEM. Data preparation, pre-processing, and statistical analysis (see Section 4.2) were performed in Python using `pandas` and `numpy` libraries. The three models introduced in Section 3 were developed in Python by using the open-source `H2O` library. The tests were executed on a 2.3GHz Intel Xeon Gold 6252N with 16 GB of memory.

5.1 Experimental Setup

After the pre-processing and cleaning phase described in Section 4.3), we obtained a dataset $\mathcal{D}$ composed of 9005 samples. Figure 1 shows, on the left, the distribution of UMJESTs observed in $\mathcal{D}$ and, on the right, the probability distribution that best fits the data, namely, beta distribution (found using Distfit library) with parameters α and β equal to 2.93 and 6.47, respectively.

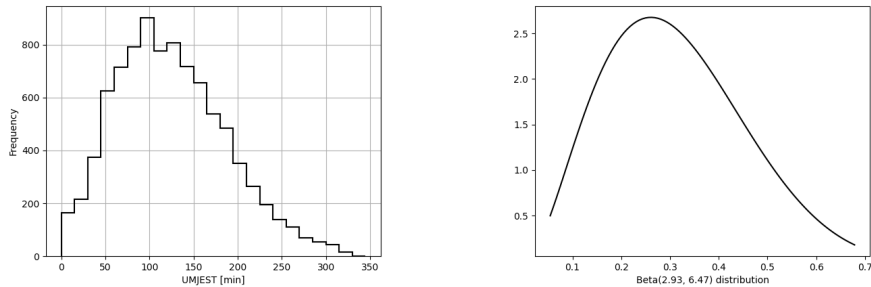


Fig. 1: The distribution of UMJESTs from $\mathcal{D}$ and the resulting probability distribution.

The models were fit on set $\mathcal{D}_{train}$ composed of 7890 samples collected between January 2019 and April 2022. As the number of entries in the available real-world industrial dataset is limited, to avoid reducing the number of samples by removing a held-out dataset and running into any bias problem, the evaluation of ML models was performed via 10-fold cross-validation on $\mathcal{D}_{train}$, a procedure normally recommended for small real-world datasets [32]. More precisely, we randomly partitioned data into 10 equally-sized parts, named folds, and we trained 10 different models on 9 parts, using for testing purposes the part that was not used for training. Finally, the models were tested on a new corpus, named $\mathcal{D}_{test}$, composed of 1937 samples, which was collected later on, between May 2022 and October 2022.

5.2 Parameter Tuning

According to H2O library, LR corresponds to the Gaussian family model in which the link function is the identity, and the density corresponds to a normal distribution. LR model was fit using the least squares approach.

We trained RF by means of the random grid search, provided by H2O, over a specific hyper-parameter space defined according to preliminary experimental trials. Specifically, the number of trees takes values in $\{20, 50, 80, 110, 140, 170, 200\}$, the sampling rate in $\{0.63, 0.70, 0.80, 1.00\}$, the maximum depth of each tree in $\{10, 15, 20, 25, 30, 35\}$, the number of columns to randomly select at each level in $\{10, 20, 40, 60, 80\}$, the number of bins to be included in the histogram and then split at the best point in $\{8, 16, 32, 64, 128, 256, 512, 1024\}$. To train GBM, we used AutoML, an automated hyper-parameter tuning method provided by H2O, which gave us slightly better results than random grid search for this specific model.

Feature importance scores are reported in Figure 2. The feature with the highest importance score is f_1 , highlighting the significant correlation between UMJESTs and the specific machine used. Moreover, f_{120} , f_{12} , and f_{63} follow closely behind with high importance scores, emphasizing the significant impact of color sets required by two consecutive jobs on UMJESTs. In addition, the high importance scores of f_{61} and f_{11} provide further evidence of the impact that changing materials between two consecutive jobs has on UMJESTs. These findings are consistent with the experience of industry practitioners, who acknowledge that washing a color group is a highly time-consuming process and depends on the specific colors involved. Similarly, changing materials between jobs has been identified as a critical factor that can significantly impact manufacturing efficiency. We trained LR, RF, and GBM with $\alpha \in \{0, 0.05, 0.10, 0.15, 0.20, 0.25\}$, using the first 120, 52, 47, and 40 features, respectively, as shown next.

5.3 Forecasting Results

Performance evaluations of the obtained models were done by measuring their quality using five statistical indicators: *mean square error* (MSE), *root mean square error* (RMSE), *mean absolute error* (MAE), and *root mean squared log error* (RMSLE).

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \tilde{y}_i)^2 \quad (1)$$

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \tilde{y}_i)^2} \quad (2)$$

$$MAE = \frac{1}{m} \sum_{i=1}^m |y_i - \tilde{y}_i| \quad (3)$$

$$RMSLE = \sqrt{\frac{1}{m} \sum_{i=1}^m (\log y_i - \log \tilde{y}_i)^2} \quad (4)$$

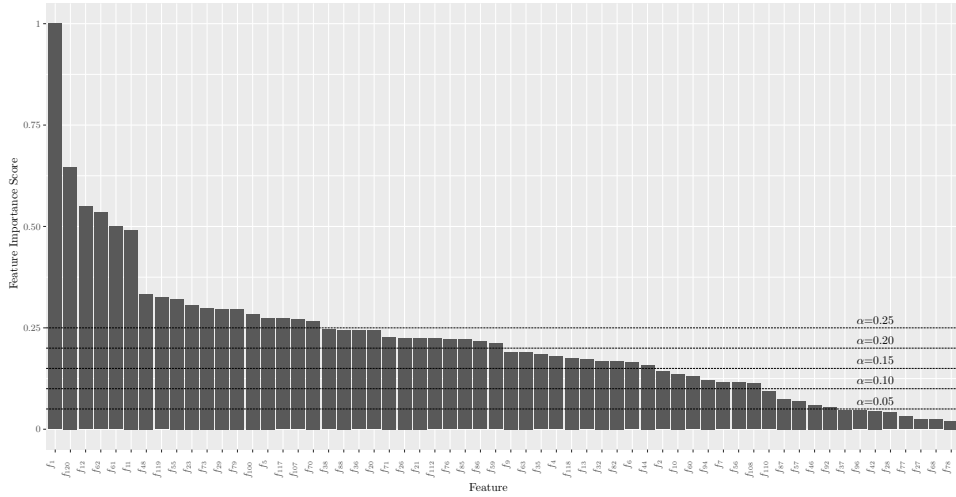


Fig. 2: Feature importance scores of the first 60 features.

As shown in Equation (1), MSE measures the average squared difference between the predicted values and the actual values, providing a measure of the overall accuracy of the model. RMSE, as depicted in (2), is the square root of the MSE. It is particularly useful when evaluating models in the same unit as the target variable. Indeed, RMSE provides a more interpretable measure compared to MSE, because it is on the same scale as the target variable. Equation (3) represents MAE, which calculates the average absolute difference between the predicted values and the actual values. MAE is not influenced by outliers to the same extent as MSE, making it a robust metric for model evaluation. Finally, Equation (4) shows RMSLE, which measures the root mean squared logarithmic difference between the predicted values and the actual values. RMSLE is commonly used when the target variable has a wide range, and it penalizes underestimations and overestimations symmetrically on a logarithmic scale.

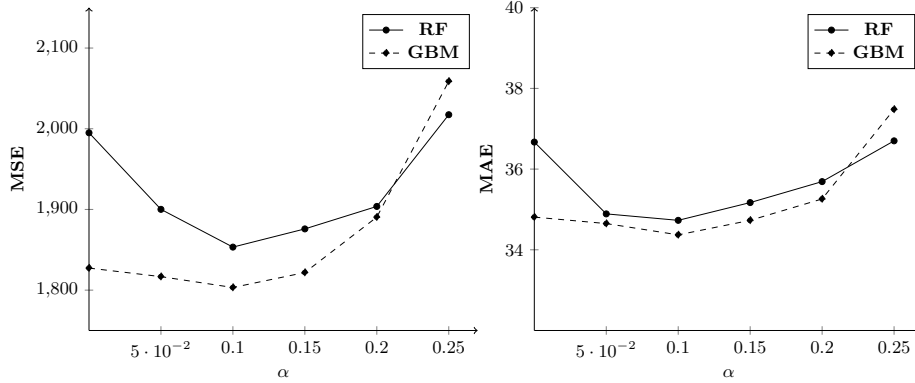
Table 3 shows the computational results of EEM, LR, RF, and GBM for different values of α . Thus, models were trained using different numbers of features, depending on the value of α , and their performances were evaluated in terms of MSE, RMSE, MAE, and RMSLE.

RF and GBM models consistently outperform the LR and EEM, with lower MSE, RMSE, and MAE values for all considered threshold parameter values. In particular, their performances increase as α increases (up to $\alpha = 0.10$), indicating that feature selection helps to simplify the models and reduce overfitting. More specifically, Table 3 highlights that setting α to 0.10 produces the best performance for both models by striking a balance between having too many features, which may lead to overfitting issues, and too few features, which may

lead to underfitting. Moreover, we can observe how the worse performance of LR is likely due to the failure of the model to describe the complex and non-linear relation between UMJESTs and the input variables.

Table 3: Computational results of EEM, LR, RF, and GBM.

Model	α	#features	MSE	RMSE	MAE	RMSLE
EEM	-	-	4080.60	61.28	46.58	-
LR	0.00	120	6422.50	80.14	58.53	-
RF	0.00	120	1995.00	44.66	36.67	0.42
GBM	0.00	120	1827.35	42.74	34.81	0.60
LR	0.05	52	4355.59	66.00	50.63	-
RF	0.05	52	1900.12	43.59	34.89	0.41
GBM	0.05	52	1816.73	42.62	34.65	0.40
LR	0.10	47	4353.31	65.98	50.61	-
RF	0.10	47	1853.33	43.05	34.73	0.40
GBM	0.10	47	1803.27	42.46	34.37	0.44
LR	0.15	40	4312.58	65.67	50.57	-
RF	0.15	40	1875.88	43.31	35.17	0.41
GBM	0.15	40	1821.83	42.68	34.73	0.40
LR	0.20	26	5065.27	71.17	55.12	-
RF	0.20	26	1903.73	43.63	35.69	0.41
GBM	0.20	26	1890.49	43.47	35.26	0.44
LR	0.25	18	5143.02	71.71	55.76	-
RF	0.25	18	2017.39	44.91	36.70	0.42
GBM	0.25	18	2058.74	45.37	37.48	0.43

Fig. 3: Average z value over the 25 instances, by varying the threshold parameter.

6 Conclusions

In this paper, we faced the problem of predicting ST in the context of real-world manufacturing environments. Although shortcomings in the ST prediction

may lead to develop inefficient schedules and represent a relevant source of gap between scheduling theory and practice, ST estimation received limited attention in the literature.

To fill this gap, we explored the use of ML regression algorithms to provide good estimations of the ST. We applied these ML algorithms to a real-world scheduling application arising in the color printing industry, where a finite set of jobs must be sequentially performed by a heterogeneous set of parallel flexographic printer machines. Specifically, we dealt with UMJSSTs which are typical in the context of flexible manufacturing systems, and are hard to predict. After data preparation, pre-processing, and statistical analysis of a real-world industrial dataset, we trained three different ML models, namely, LR, RF, and GBM. Computational experiments assessed the effectiveness of GBM approach, which obtained the best performance overall, immediately followed by RF. For both models, the mean squared error on the predicted UMJESTs is less than a third of LR and less than half of that of a tailored heuristic approach.

The accuracy of the obtained predictions shows the effectiveness of the proposed ML approach in UMJESTs evaluation. The approach is significant and valuable because the employed ML algorithms can be effectively used to predict STs across a range of scheduling scenarios. Furthermore, the approach can also be easily integrated into scheduling algorithms, presenting a promising opportunity to enhance the precision and accuracy in scheduling processes. In particular, as a possible topic for future research, we are interested in embedding this ML approach, or even more sophisticated ones (with a particular focus on deep learning techniques), into scheduling algorithms to improve their accuracy and narrow the gap between scheduling theory and practice.

Acknowledgment

We acknowledge financial support from Istituto Stampa s.r.l. (Italy). The work originates from the daily activity of Istituto Stampa s.r.l., a company whose headquarters are located in Reggio Emilia (Italy). The company operates in the field of packaging industry by producing and printing packaging materials for food products since 1933.

The authors also gratefully acknowledge financial support under the National Recovery and Resilience Plan (NRRP), Mission 04 Component 2 Investment 1.5–NextGenerationEU, Call for tender n. 3277, Award n. 0001052.

Bibliography

- [1] A. Allahverdi and H. M. Soroush. The significance of reducing setup times/setup costs. *European Journal of Operational Research*, 187:978–984, 2008.
- [2] A. Allahverdi. The third comprehensive survey on scheduling problems with setup times/costs. *European Journal of Operational Research*, 246:345–378, 2015.

- [3] S. C. Kim and P. M. Bobrowski. Scheduling jobs with uncertain setup times and sequence dependency. *Omega*, 25:437–447, 1997.
- [4] I. Sabuncuoglu and S. Goren. Hedging production schedules against uncertainty in manufacturing environment with a review of robustness and stability research. *International Journal of Computer Integrated Manufacturing*, 22:138–157, 2009.
- [5] A. Allahverdi. A survey of scheduling problems with uncertain interval/bounded processing/setup times. *Journal of Project Management*, 7: 255–264, 2022.
- [6] L. C. R. Soares and M. A. M. Carvalho. Biased random-key genetic algorithm for scheduling identical parallel machines with tooling constraints. *European Journal of Operational Research*, 285:955–964, 2020.
- [7] M. Iori, A. Locatelli, and M. Locatelli. Scheduling of parallel print machines with sequence-dependent setup costs: A real-world case study. In *IFIP International Conference on Advances in Production Management Systems*, pages 637–645. Springer, 2021.
- [8] M. Iori, A. Locatelli, and M. Locatelli. A GRASP for a real-world scheduling problem with unrelated parallel print machines and sequence-dependent setup times. *International Journal of Production Research*, pages 1–19, 2022 (forthcoming).
- [9] M. Iori, A. Locatelli, M. Locatelli, and J. J. Salazar-González. Tool switching problems in the context of overlay printing with multiple colours. In *Combinatorial Optimization*, pages 260–271. Springer, 2022.
- [10] E. González-Neira, J. Montoya-Torres, and D. Barrera. Flow-shop scheduling problem under uncertainties: Review and trends. *International Journal of Industrial Engineering Computations*, 8:399–426, 2017.
- [11] A. Aydilek, H. Aydilek, and A. Allahverdi. Increasing the profitability and competitiveness in a production environment with random and bounded setup times. *International Journal of Production Research*, 51:106–117, 2013.
- [12] A. Aydilek, H. Aydilek, and A. Allahverdi. Production in a two-machine flowshop scheduling environment with uncertain processing and setup times to minimize makespan. *International Journal of Production Research*, 53: 2803–2819, 2015.
- [13] A. Allahverdi and M. Allahverdi. Two-machine no-wait flowshop scheduling problem with uncertain setup times to minimize maximum lateness. *Computational and Applied Mathematics*, 37:6774–6794, 2018.
- [14] M. Allahverdi. An improved algorithm to minimize the total completion time in a two-machine no-wait flow-shop with uncertain setup times. *Journal of Project Management*, 7:1–12, 2022.
- [15] A. D. Yimer and K. Demirli. Fuzzy scheduling of job orders in a two-stage flowshop with batch-processing machines. *International Journal of Approximate Reasoning*, 50:117–137, 2009.
- [16] C. C. Lu, S. W. Lin, and K. C. Ying. Robust scheduling on a single machine to minimize total flow time. *Computers & Operations Research*, 39: 1682–1691, 2012.

- [17] M. Naderi-Beni, E. Ghobadian, S. Ebrahimnejad, and R. Tavakkoli-Moghaddam. Fuzzy bi-objective formulation for a parallel machine scheduling problem with machine eligibility restrictions and sequence-dependent setup times. *International Journal of Production Research*, 52:5799–5822, 2014.
- [18] J. Behnamian and S. M. T. Fatemi Ghomi. Multi-objective fuzzy multiprocessor flowshop scheduling. *Applied Soft Computing Journal*, 21:139–148, 2014.
- [19] C. Y. Cheng, P. Pourhejazy, K. C. Ying, S. F. Li, and C. W. Chang. Learning-based metaheuristic for scheduling unrelated parallel machines with uncertain setup times. *IEEE Access*, 8:74065–74082, 2020.
- [20] G. Chryssolouris. *Manufacturing systems: theory and practice*. Springer Science & Business Media, 2013.
- [21] Y. Crama. Combinatorial optimization models for production scheduling in automated manufacturing systems. *European Journal of Operational Research*, 99:136–153, 1997.
- [22] G. Laporte, J. J. Salazar-González, and F. Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36:37–45, 2004.
- [23] D. C. Montgomery, E. A. Peck, and G. G. Vining. *Introduction to linear regression analysis*. John Wiley & Sons, 2021.
- [24] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and regression trees*. Routledge, 2017.
- [25] A. Natekin and A. Knoll. Gradient boosting machines, a tutorial. *Frontiers in Neurorobotics*, 7, 2013.
- [26] M. Kuhn and K. Johnson. *Applied predictive modeling*. Springer, 2013.
- [27] C. Lewis-Beck and M. Lewis-Beck. *Applied regression: An introduction*, volume 22 of *Quantitative Applications in the Social Sciences*. Sage publications, 2015.
- [28] L. Breiman. Random forests. *Machine learning*, 45:5–32, 2001.
- [29] T. Hastie, R. Tibshirani, and J. H. Friedman. *The elements of statistical learning: data mining, inference, and prediction*, volume 2 of *Springer Series in Statistics*. Springer, 2009.
- [30] H. Kipphan. *Handbook of print media: technologies and production methods*. Springer Science & Business Media, 2001.
- [31] N. C. Schwertman and R. de Silva. Identifying outliers with sequential fences. *Computational Statistics and Data Analysis*, 51:3800–3810, 2007.
- [32] P. Refaeilzadeh, L. Tang, and H. Liu. *Cross-Validation*, pages 532–538. Springer, 2009.